

A decorative graphic on the left side of the slide, consisting of a network of thin, gold-colored lines and small circles, resembling a circuit board or a neural network diagram.

Bitwise Operators

COMP211 Connor McMahon

Reminders

- Lab 0
 - Make sure you accept the Lab 0 assignment today!!!
 - The lab is due on Tuesday
- HW2 is due today
- There is no class or office hours on Monday (Labor Day)

Bitwise Operators

Fill in the blanks using the provided options. n is an integer.

Answer bank

n
 $\sim n$
 $0b\ 0\dots 0$
 $0b\ 1\dots 1$

n	$ $	_____	$=$	n
n	$ $	_____	$=$	n
n	$ $	_____	$=$	$0b1\dots 1$
n	$ $	_____	$=$	$0b1\dots 1$

n	$\&$	_____	$=$	n
n	$\&$	_____	$=$	n
n	$\&$	_____	$=$	$0b0\dots 0$
n	$\&$	_____	$=$	$0b0\dots 0$

n	$\wedge$	_____	$=$	n
n	$\wedge$	_____	$=$	$0b0\dots 0$
n	$\wedge$	_____	$=$	$0b1\dots 1$

Bitwise Operators

Fill in the blanks using the provided options. n is an integer.

Answer bank

	n
	~n
0b	0...0
0b	1...1

n		__0b0...0__	=	n
n		_____n_____	=	n
n		_____	=	0b1...1
n		_____	=	0b1...1

n	&	__0b1...1__	=	n
n	&	_____	=	n
n	&	_____	=	0b0...0
n	&	_____	=	0b0...0

n	^	-----	=	n
n	^	-----	=	0b0...0
n	^	-----	=	0b1...1

Bitwise Operators



Fill in the blanks using the provided options. n is an integer.

Answer bank

n
 $\sim n$
 $0b\ 0\dots 0$
 $0b\ 1\dots 1$

$$\begin{aligned} n \mid 0b0\dots 0 &= n \\ n \mid n &= n \\ n \mid 0b1\dots 1 &= 0b1\dots 1 \\ n \mid \sim n &= 0b1\dots 1 \end{aligned}$$
$$\begin{aligned} n \& 0b1\dots 1 &= n \\ n \& n &= n \\ n \& 0b0\dots 0 &= 0b0\dots 0 \\ n \& \sim n &= 0b0\dots 0 \end{aligned}$$
$$\begin{aligned} n \wedge 0b0\dots 0 &= n \\ n \wedge n &= 0b0\dots 0 \\ n \wedge \sim n &= 0b1\dots 1 \end{aligned}$$

Shifting

Left Shift

- Syntax
 - `number << n`
- Effect:
 - Shifts all bits to the left by `n` places.
 - Fills the rightmost `n` bits with 0s.
- Equivalent to multiplying the number by 2^n .

Left Shift

```
uint8_t a = 5;  
uint8_t b = a << 1;  
uint8_t c = a << 2;  
uint8_t d = a << 3;
```

a = 0b0000_0101 = 5
b = 0b0000_1010 = 10 = 5 x 2¹
c = 0b0001_0100 = 20 = 5 x 2²
d = 0b0010_1000 = 40 = 5 x 2³

Logical Right Shift

- Used for **unsigned** numbers
- Syntax
 - `number >> n`
- Effect:
 - Shifts all bits to the right by `n` places.
 - Fills the leftmost `n` bits with 0s.
- Equivalent to integer division by 2^n , truncating any decimal part.

Logical Right Shift

```
uint8_t a = 5;  
uint8_t b = a >> 1;  
uint8_t c = a >> 2;  
uint8_t d = a >> 3;
```

```
a = 0b0000_0101 = 5  
b = 0b0000_0010 = 2  
c = 0b0000_0001 = 1  
d = 0b0000_0000 = 0
```

Arithmetic Right Shift

- Used for **signed** numbers
- Syntax
 - `number >> n`
- Effect:
 - Shifts all bits to the right by `n` places.
 - Fills the leftmost `n` bits with a copy of the msb.
- Equivalent to integer division by 2^n , rounding towards negative infinity.
- In C, unsigned right shifts are logical, while signed right shifts are implementation-defined (usually arithmetic).

Arithmetic Right Shift on a Positive Number

```
int8_t a = 5;  
int8_t b = a >> 1;  
int8_t c = a >> 2;  
int8_t d = a >> 3;
```

```
a = 0b0000_0101 = 5  
b = 0b0000_0010 = 2  
c = 0b0000_0001 = 1  
d = 0b0000_0000 = 0
```

Arithmetic Right Shift on a Negative Number

```
int8_t a = -5;  
int8_t b = a >> 1;  
int8_t c = a >> 2;  
int8_t d = a >> 3;
```

```
a = 0b1111_1011 = -5  
b = 0b1111_1101 = -3  
c = 0b1111_1110 = -2  
d = 0b1111_1111 = -1
```

What does this program output?

```
#include <stdint.h>
#include <stdio.h>

int main(void) {
    uint8_t b = 2;
    int8_t c = 10;
    int8_t d = -2;
    printf("%d\n", b >> 1);
    printf("%d\n", b << 5);
    printf("%d\n", c >> 2);
    printf("%d\n", d << 1);
    printf("%d\n", d >> 1);
    printf("%d\n", (uint8_t)(d >> 1));
    printf("%d\n", (uint8_t)(d) >> 1);
}
```

What does this program output?



```
#include <stdint.h>
#include <stdio.h>

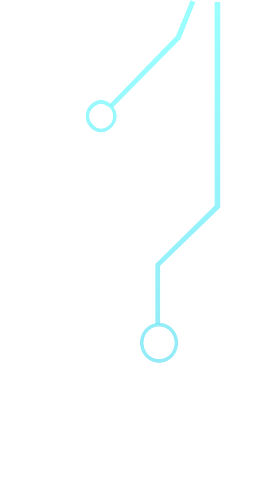
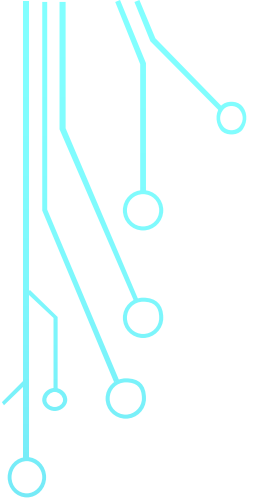
int main(void) {
    uint8_t b = 2;
    int8_t c = 10;
    int8_t d = -2;
    printf("%d\n", b >> 1);
    printf("%d\n", b << 5);
    printf("%d\n", c >> 2);
    printf("%d\n", d << 1);
    printf("%d\n", d >> 1);
    printf("%d\n", (uint8_t)(d >> 1));
    printf("%d\n", (uint8_t)(d) >> 1);
}
```

1
64
2
-4
-1
255
127

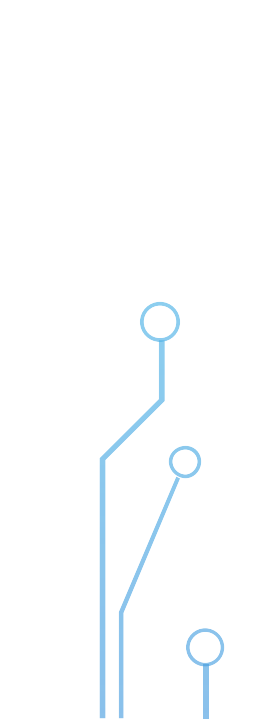
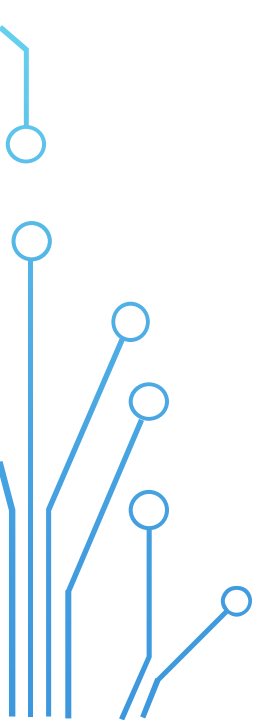
Masks

Masks

- A **mask** is a binary pattern used to **select (get), set, clear, or toggle specific bits** within a number.
- **Select**(get) = read
- **Set** = make 1
- **Clear** = make 0
- **Toggle** = flip from 0 to 1 or 1 to 0



Select (get)



get_bit

- Let's write a function `getBit` that will return whether a given bit is on (1) or off (0)

```
#include <stdint.h>
#include <stdio.h>

int8_t get_bit(int8_t num, int8_t bit);

int main(void) {
    printf("%d\n", get_bit(9, 1));
    printf("%d\n", get_bit(9, 3));
    printf("%d\n", get_bit(18, 4));
    return 0;
}
```

Expected Output

0

1

1

get_bit

- Let's write a function `getBit` that will return whether a given bit is on (1) or off (0)

```
#include <stdint.h>
#include <stdio.h>

int8_t get_bit(int8_t num, int8_t bit) {
    int8_t mask = 1 << bit;
    int8_t isolate_bit = num & mask;
    return (isolate_bit != 0);
}

int main(void) {
    printf("%d\n", get_bit(8, 1));
    printf("%d\n", get_bit(8, 3));
    printf("%d\n", get_bit(16, 4));
    return 0;
}
```

Example Inputs

	num = 9 bit = 1	num = 18 bit = 4
num		
mask		
isolate_bit		
RV		

get_bit

- Let's write a function `getBit` that will return whether a given bit is on (1) or off (0)

```
#include <stdint.h>
#include <stdio.h>

int8_t get_bit(int8_t num, int8_t bit) {
    int8_t mask = 1 << bit;
    int8_t isolate_bit = num & mask;
    return (isolate_bit != 0);
}

int main(void) {
    printf("%d\n", get_bit(8, 1));
    printf("%d\n", get_bit(8, 3));
    printf("%d\n", get_bit(16, 4));
    return 0;
}
```

Example Inputs

	num = 9 bit = 1	num = 18 bit = 4
num	0b0000_1001	0b0001_0010
mask	0b0000_0010	0b0001_0000
isolate_bit	0b0000_0000	0b0001_0000
RV	0	1

Set

set_bit_2

- Let's write a function called `set_bit_2` that will set bit 2 of the input `num` to 1

```
int8_t set_bit_2(int8_t num);

int main(void) {
    printf("%d\n", set_bit_2(10));
    printf("%d\n", set_bit_2(8));
    printf("%d\n", set_bit_2(12));
    return 0;
}
```

```
10 = 0b0000_1010 -> 0b0000_1110
 8 = 0b0000_1000 -> 0b0000_1100
12 = 0b0000_1100 -> 0b0000_1100
```

set_bit_2

- Let's write a function called `set_bit_2` that will set bit 2 of the input `num` to 1

```
int8_t set_bit_2(int8_t num) {  
    int8_t mask = 0b00000100;  
    return (mask | num);  
}  
  
int main(void) {  
    printf("%d\n", set_bit_2(10));  
    printf("%d\n", set_bit_2(8));  
    printf("%d\n", set_bit_2(12));  
    return 0;  
}
```

set_bit

- Let's write a function `set_bit` that will set bit `i` of `num` to 1

```
int8_t set_bit(int8_t num, int8_t i);  
  
int main(void) {  
    printf("%d\n", set_bit(10, 1));  
    printf("%d\n", set_bit(8, 0));  
    printf("%d\n", set_bit(12, 4));  
    return 0;  
}
```

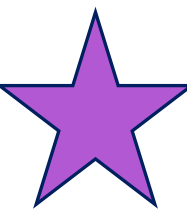
```
10 = 0b0000_1010 -> 0b0000_1010 = 10  
 8 = 0b0000_1000 -> 0b0000_1001 = 9  
12 = 0b0000_1100 -> 0b0001_1100 = 28
```

set_bit

- Let's write a function `set_bit` that will set bit `i` of `num` to 1

```
int8_t set_bit(int8_t num, int8_t i) {  
  
  
  
  
  
  
  
  
  
}  
  
int main(void) {  
    printf("%d\n", set_bit(10, 1));  
    printf("%d\n", set_bit(8, 0));  
    printf("%d\n", set_bit(12, 4));  
    return 0;  
}
```

set_bit



- Let's write a function `set_bit` that will set bit `i` of `num` to 1

```
int8_t set_bit(int8_t num, int8_t i) {  
    int8_t mask = 1 << i;  
    return (mask | num);  
}
```

```
int main(void) {  
    printf("%d\n", set_bit(10, 1));  
    printf("%d\n", set_bit(8, 0));  
    printf("%d\n", set_bit(12, 4));  
    return 0;  
}
```

Clear

clear_bit2

- Let's write a function, `clear_bit2`, that takes an integer as input, sets bit 2 to 0, and returns the modified result.
- First, let's write some test cases:

```
#define NUM_TESTS 3

int8_t clear_bit2(int8_t number);

int main() {
    int8_t tests[NUM_TESTS] = {14, 12, 8};
    for (int i = 0; i < NUM_TESTS; i++) {
        printf("Original: %d, Modified: %d\n",
            tests[i], clear_bit2(tests[i]));
    }
    return 0;
}

...
```

14 = 0b0000_1110	➡	0b0000_1010 = 10
12 = 0b0000_1100		0b0000_1000 = 8
8 = 0b0000_1000		0b0000_1000 = 8

Program Output:

```
Original: 14, Modified: 10
Original: 12, Modified: 8
Original: 8, Modified: 8
```

clear_bit2

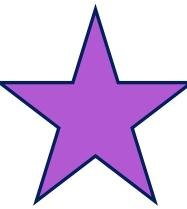
```
#define NUM_TESTS 3

int8_t clear_bit2(int8_t number);

int main() {
    int8_t tests[NUM_TESTS] = {14, 12, 8};
    for (int i = 0; i < NUM_TESTS; i++) {
        printf("Original: %d, Modified: %d\n", tests[i], clear_bit2(tests[i]));
    }
    return 0;
}

int8_t clear_bit2(int8_t number) {
    int8_t mask = -----;
    return -----;
}
```

clear_bit2



```
#define NUM_TESTS 3

int8_t clear_bit2(int8_t number);

int main() {
    int8_t tests[NUM_TESTS] = {14, 12, 8};
    for (int i = 0; i < NUM_TESTS; i++) {
        printf("Original: %d, Modified: %d\n", tests[i], clear_bit2(tests[i]));
    }
    return 0;
}

int8_t clear_bit2(int8_t number) {
    int8_t mask = ~(0b100);
    return number & mask;
}
```

clear_bit

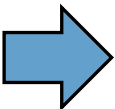
- Let's write a function that will clear bit *i* of the input *num*

```
#define NUM_TESTS 3

int8_t clear_bit(int8_t num, int8_t i);

int main() {
    int tests[NUM_TESTS] = {14, 12, 8};
    for (int i = 0; i < NUM_TESTS; i++) {
        printf("Original: %d, Modified: %d\n", tests[i], clear_bit(tests[i], i + 2));
    }
    return 0;
}
```

Program Output:

14 = 0b0000_1110		0b0000_1010 = 10
12 = 0b0000_1100		0b0000_0100 = 8
8 = 0b0000_1000		0b0000_1000 = 8

```
Original: 14, Modified: 10
Original: 12, Modified: 4
Original: 8, Modified: 8
```

clear_bit

- Let's write a function that will clear bit *i* of the input *num*

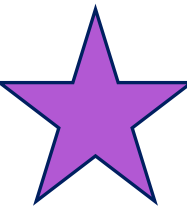
```
#define NUM_TESTS 3

int8_t clear_bit(int8_t num, int8_t i) {

}

int main() {
    int tests[NUM_TESTS] = {14, 12, 8};
    for (int i = 0; i < NUM_TESTS; i++) {
        printf("Original: %d, Modified: %d\n", tests[i], clear_bit(tests[i], i + 2));
    }
    return 0;
}
```

clear_bit



- Let's write a function that will clear bit *i* of the input *num*

```
#define NUM_TESTS 3
```

```
int8_t clear_bit(int8_t num, int8_t i) {  
    int8_t mask = ~(1 << i);  
    return num & mask;  
}
```

```
int main() {  
    int tests[NUM_TESTS] = {14, 12, 8};  
    for (int i = 0; i < NUM_TESTS; i++) {  
        printf("Original: %d, Modified: %d\n", tests[i], clear_bit(tests[i], i + 2));  
    }  
    return 0;  
}
```

	num = 14 i = 2	num = 8 i = 4
num	0b0000_1110	0b0000_1000
(1 << i)	0b0000_0100	0b0001_0000
mask	0b1111_1011	0b1110_1111
RV	0b0000_1010	0b0000_1000